



19

Navigating Between Screens

The focus of this chapter is on navigating between the screens that make up your React Native application. Navigation in native apps is slightly different than navigation on web apps: mainly because there isn't any notion of a URL that the user is aware of. In prior versions of React Native, there were primitive navigator components that you could use to control the navigation between screens. There were a number of challenges with these components that resulted in more code to accomplish basic navigation tasks. For example, initial navigation components, like `Navigator` and `NavigatorIOS`, were complex to implement and lacked features, leading to performance issues and inconsistency across platforms.

More recent versions of **React Native** encourage you to use the `react-navigation` package, which will be the focus of this chapter, even though there are several other options. You'll learn about navigation basics, passing parameters to screens, changing header content, using tab and drawer navigation,

and handling state with navigation. Also, we'll take a look at a modern navigation approach called file-based navigation.

We'll cover the following topics in this chapter:

- The basics of navigation
- Route parameters
- The navigation header
- Tab and drawer navigation
- File-based navigation

Technical requirements

You can find the code files for this chapter on GitHub at

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter19>.

The basics of navigation

Navigation in React Native is crucial because it manages the transition between different screens in an app. It improves user experience by organizing the app's flow logically, allowing users to intuitively understand how to access features and information. Effective navigation makes an app feel quick and responsive, reducing frustration and increasing user engagement. It also supports the app's architecture, making it easier to scale and maintain by clearly defining how components are

linked and interact. Without proper navigation, an app can become confusing and difficult to use, significantly impacting its success and user retention. This section will guide you through setting up navigation in your app by creating a small app where you can navigate between screens.

Let's start off with the basics of moving from one page to another using the `react-navigation` package.

Before starting, you should install the `react-navigation` package to a fresh project and some additional dependencies related to the example:

```
npm install @react-navigation/native
```

Then, install native dependencies using `expo`:

```
npx expo install react-native-screens react-native-safe-area-context
```

The preceding installation steps will be required for each example in this chapter, but we need to add one more package related to the stack navigator:

```
npm install @react-navigation/native-stack
```

Now, we are ready to develop navigation. Here's what the `App` component looks like:

```
import Home from "../Home";
import Settings from "../Settings";
const Stack = createNativeStackNavigator<RootStackParamList>();
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={Home} />
        <Stack.Screen name="Settings" component={Settings} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

`createNativeStackNavigator()` is a function that sets up your navigation. It returns an object with two properties, the `Screen` and `Navigator` components, that are used for configuring the stack navigator.

The first argument to this function maps to the screen components that can be navigated. The second argument is for more general navigation options: in this case, you're telling the navigator that the homepage should be the default screen component that's rendered. The `<NavigationContainer>` component

is necessary so that the screen components get all of the navigation properties that they need.

Here's what the `Home` component looks like:

```
type Props = NativeStackScreenProps<RootStackParamList>;
export default function Home({ navigation }: Props) {
  return (
    <View style={styles.container}>
      <StatusBar barStyle="dark-content" />
      <Text>Home Screen</Text>
      <Button
        title="Settings"
        onPress={() => navigation.navigate("Settings")}
      />
    </View>
  );
}
```

This is your typical functional React component. You can use a class-based component here, but there's no need since there is no state or life cycle methods. It renders a `View` component where the container style is applied.

This is followed by a `Text` component that labels the screen followed by a `Button` component. A screen can be anything you want: it's just a regular React Native component. The navi-

gator component handles the routing and the transitions between screens for you.

The `onPress` handler for this button navigates to the **Settings** screen when clicked. This is done by calling `navigation.navigate('Settings')`. The `navigation` property is passed to your `screen` component by `react-navigation` and contains all of the routing functionality you need. In contrast to working with URLs in React web apps, here you call navigator API functions and pass them the names of screens.

To get type safe environment in navigation, we need to define a type called `RootStackParamList` that contains all the information about our routes. We use it together with `NativeStackScreenProps` to define route `Props`. This is what `RootStackParamList` looks like:

```
export type RootStackParamList = {  
  Home: undefined;  
  Settings: undefined;  
};
```

We pass `undefined` to each route because we don't have any parameters on routes. As a result, we can call `navigation.navigate()` only with `Settings` or `Home`.

Let's take a look at the `Settings` component:

```
type Props = NativeStackScreenProps<RootStackParamList>;
export default function Settings({ navigation }: Props) {
  return (
    <View style={styles.container}>
      <StatusBar barStyle="dark-content" />
      <Text>Settings Screen</Text>
      <Button title="Home" onPress={() => navigation.navigate("Home")} />
    </View>
  );
}
```

This component is just like the `Home` component, except with different text, and when the button is clicked, you're taken back to the **Home screen**.

This is what the **Home screen** looks like:

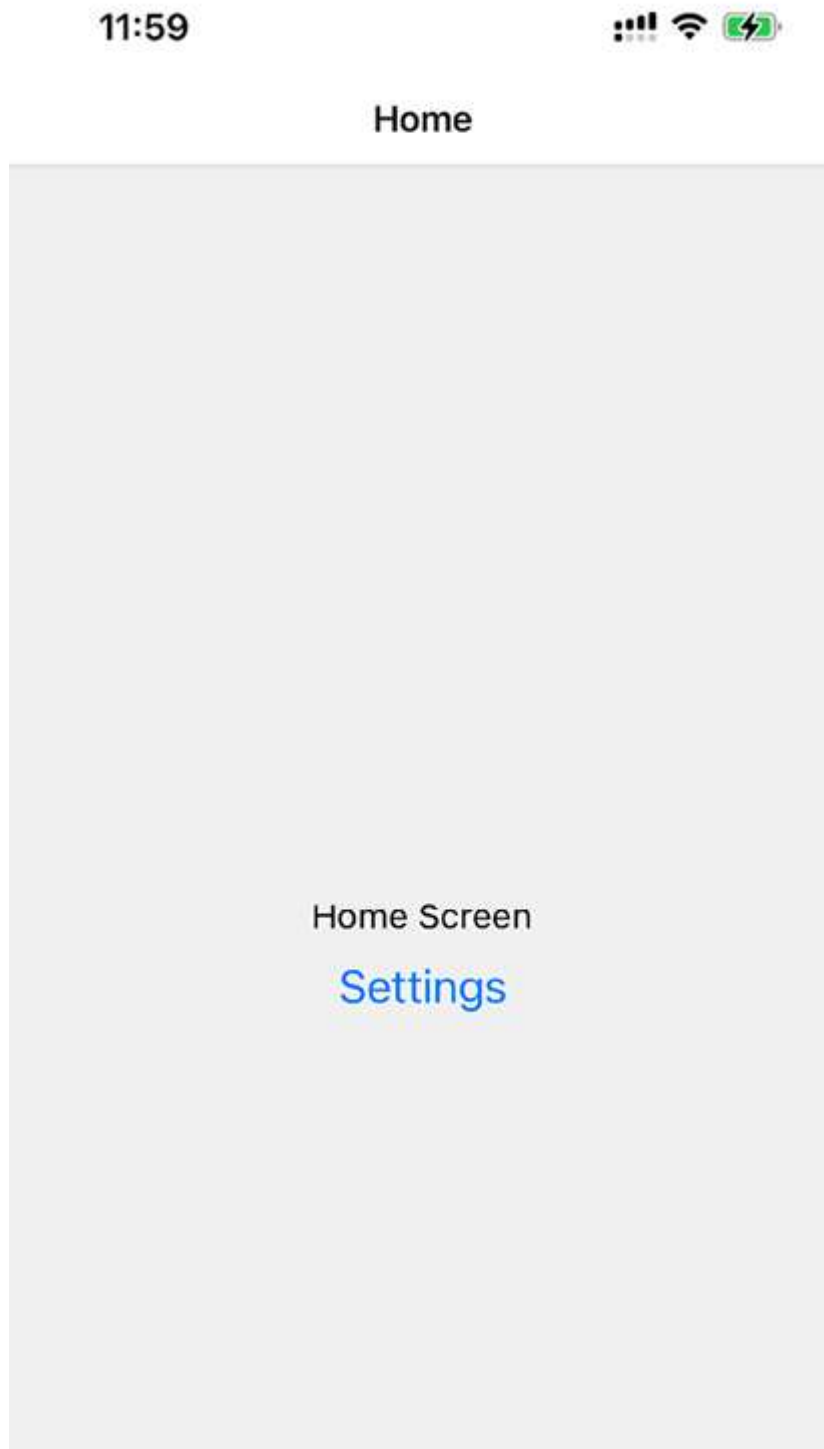




Figure 19.1: The Home screen

If you click the **Settings** button, you'll be taken to the **Settings screen**, which looks like this:

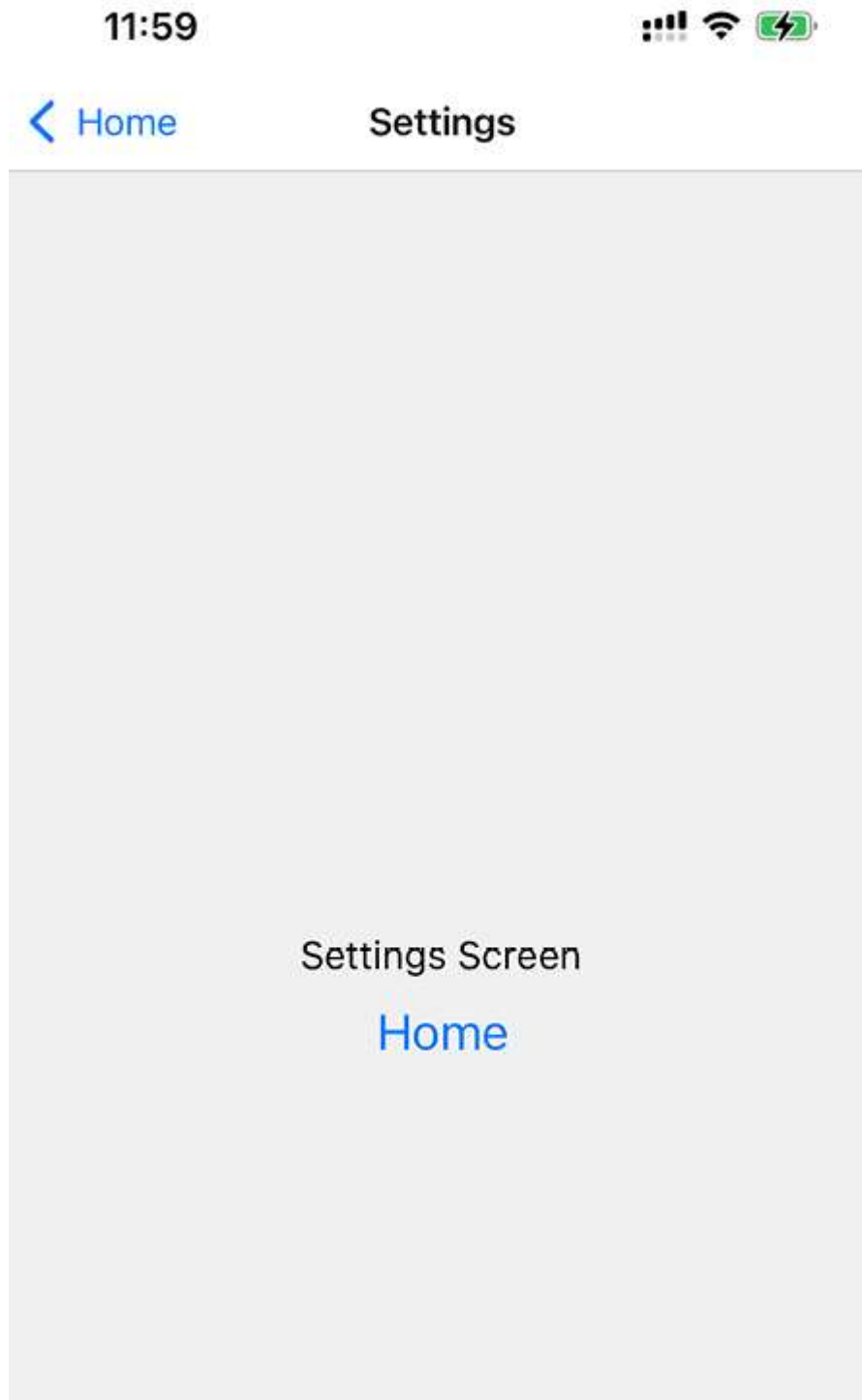




Figure 19.2: The Settings screen

This screen looks almost identical to the **Home screen**. It has different text and a different button that will take you back to the **Home screen** when clicked. However, there's another way to get back to the **Home screen**. Take a look at the top of the screen, where you'll notice a white navigation bar. On the left side of the navigation bar, there's a back arrow. This works just like the back button in a web browser and will take you back to the previous screen. What's nice about react-navigation is that it takes care of rendering this navigation bar for you.



With this navigation bar in place, you don't have to worry about how your layout styles impact the status bar. You only need to worry about the layout of each of your screens.



If you run this app on Android, you'll see the same back button in the navigation bar. But you can also use the standard back button found outside of the app on most Android devices.

In the next section, you'll learn how to pass parameters to your routes.

Route parameters

When you develop React web applications, some of your routes have **dynamic data** in them. For example, you can link to a details page, and within that URL, you'll have some sort of identifier. The component then has what it needs to render specific detailed information. The same concept exists within `react-navigation`. Instead of just specifying the name of the screen that you want to navigate to, you can pass along additional data.

Let's take a look at route parameters in action.

We'll start with the `App` component:

```
const Stack = createNativeStackNavigator<RootStackParamList>();  
export default function App() {  
  return (  
    <NavigationContainer>
```

```
    <Stack.Navigator>
      <Stack.Screen name="Home" component={Home} />
      <Stack.Screen name="Details" component={Details} />
    </Stack.Navigator>
  </NavigationContainer>
);
}
```

This looks just like the example in the *The basics of navigation* section, except instead of a `Settings` page, there's a `Details` page. This is the page that you want to pass data to dynamically so that it can render the appropriate information.

To enable TypeScript for our routes, need to define `RootStackParamList`:

```
export type RootStackParamList = {
  Home: undefined;
  Details: { title: string };
};
```

Next, let's take a look at the `Home` screen component:

```
type Props = NativeStackScreenProps<RootStackParamList, "Home">;
export default function Home({ navigation }: Props) {
  return (
    <View style={styles.container}>
```

```
<StatusBar barStyle="dark-content" />
<Text>Home Screen</Text>
<Button
  title="First Item"
  onPress={() => navigation.navigate("Details", { title: "First Item" })}
/>
<Button
  title="Second Item"
  onPress={() => navigation.navigate("Details", { title: "Second Item" })}
/>
<Button
  title="Third Item"
  onPress={() => navigation.navigate("Details", { title: "Third Item" })}
/>
</View>
);
}
```

The `Home` screen has three `Button` components, and each navigates to the `Details` screen. Note that in the `navigation.navigate()` calls, in addition to the screen name, each has a second argument. These arguments are objects that contain specific data, which is passed to the `Details` screen.

Next, let's take a look at the `Details` screen and see how it consumes these route parameters:

```
type Props = NativeStackScreenProps<RootStackParamList, "Details">;
export default function ({ route }: Props) {
  const { title } = route.params;
  return (
    <View style={styles.container}>
      <StatusBar barStyle="dark-content" />
      <Text>{title}</Text>
    </View>
  );
}
```

Although this example is only passing one `title` parameter, you can pass as many parameters to the screen as you need to. You can access these parameters using the `params` value of the `route` prop to look up the value.

Here's what the **Home screen** looks like when rendered:

12:01



Home

Home Screen

First Item

Second Item

Third Item



Figure 19.3: The Home screen

If you click on the **First Item** button, you'll be taken to the **Details** screen that is rendered using the route parameter data:

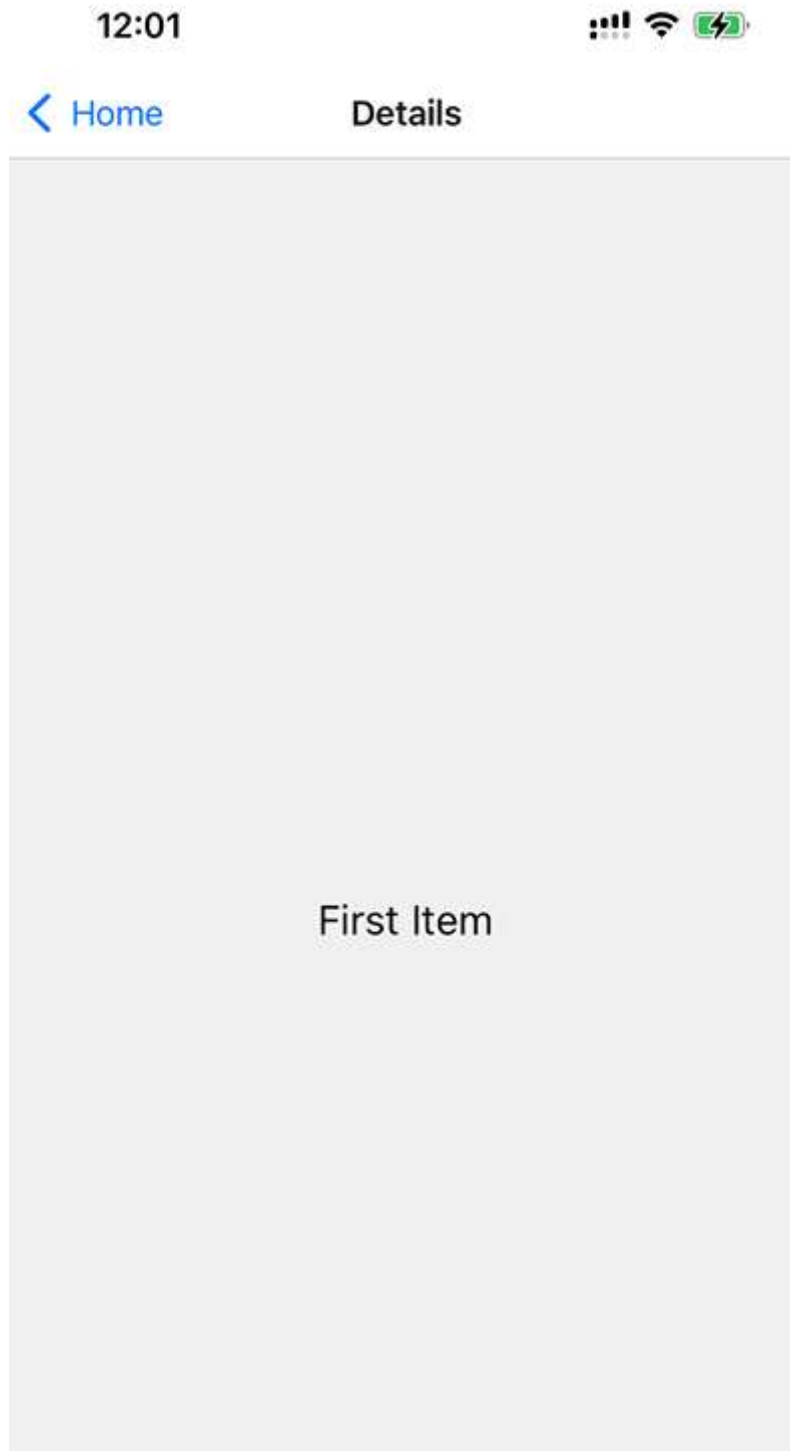




Figure 19.4: The Details screen

You can click the back button in the navigation bar to get back to the **Home** screen. If you click on any of the other buttons on the **Home screen**, you'll be taken back to the **Details** screen with updated data. Route parameters are necessary to avoid having to write duplicate components. You can think of passing parameters to `navigator.navigate()` as passing props to a React component.

In the following section, you'll learn how to populate navigation section headers with content.

The navigation header

The navigation bars that you've created so far in this chapter have been sort of plain. That's because you haven't configured them to do anything, so `react-navigation` will just render a plain bar with a back button. Each screen component that you create can configure specific navigation header content.

Let's build on the example discussed in the `Route` parameters section, which used buttons to navigate to a details page.

The `App` component has major updates, so let's take a look at it:

```
const Stack = createNativeStackNavigator<RoutesParams>();
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={Home} />
        <Stack.Screen
          name="Details"
          component={Details}
          options={({ route }) => ({
            headerRight: () => {
              return (
                <Button
                  title="Buy"
                  onPress={() => {}}
                  disabled={route.params.stock === 0}
                />
              );
            },
          })}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

```
);  
}
```

The `Screen` component accepts the `options` prop as an object or function to provide additional screen properties.

The `headerRight` option is used to add a `Button` component to the right side of the navigation bar. This is where the `stock` parameter comes into play. If this value is `0` because there isn't anything in `stock`, you want to disable the `Buy` button.

In our case, we pass `options` as a function and read the `stock` screen params to disable the button. This is one of several ways to pass options to the `Screen` component. We'll apply another way to the `Details` component.

To understand how the stock props have been passed, take a look at the `Home` component here:

```
type Props = NativeStackScreenProps<RoutesParams, "Home">;  
export default function Home({ navigation }: Props) {  
  return (  
    <View style={styles.container}>  
      <StatusBar barStyle="dark-content" />  
      <Button  
        title="First Item"  
        onPress={() =>
```

```
        navigation.navigate("Details", {
          title: "First Item",
          content: "First Item Content",
          stock: 1,
        })
      }
    />
    ...
  </View>
);
}
```

The first thing to note is that each button is passing more route parameters to the `Details` component: `content` and `stock`. You'll see why in a moment.

Next, let's take a look at the `Details` component:

```
type Props = NativeStackScreenProps<RoutesParams, "Details">;
export default function Details({ route, navigation }: Props) {
  const { content, title } = route.params;
  React.useEffect(() => {
    navigation.setOptions({ title });
  }, []);
  return (
    <View style={styles.container}>
      <StatusBar barStyle="dark-content" />
      <Text>{content}</Text>
    </View>
  );
}
```

```
);  
}
```

This time, the `Details` component renders the content `route` parameter. As with the `App` component, we add additional options to the screen. In this case, we update `screen` options using the `navigation.setOptions()` method. To customize the header, we can also add a title to that screen via the `App` component.

Let's see how all of this works now, starting with the **Home** screen:

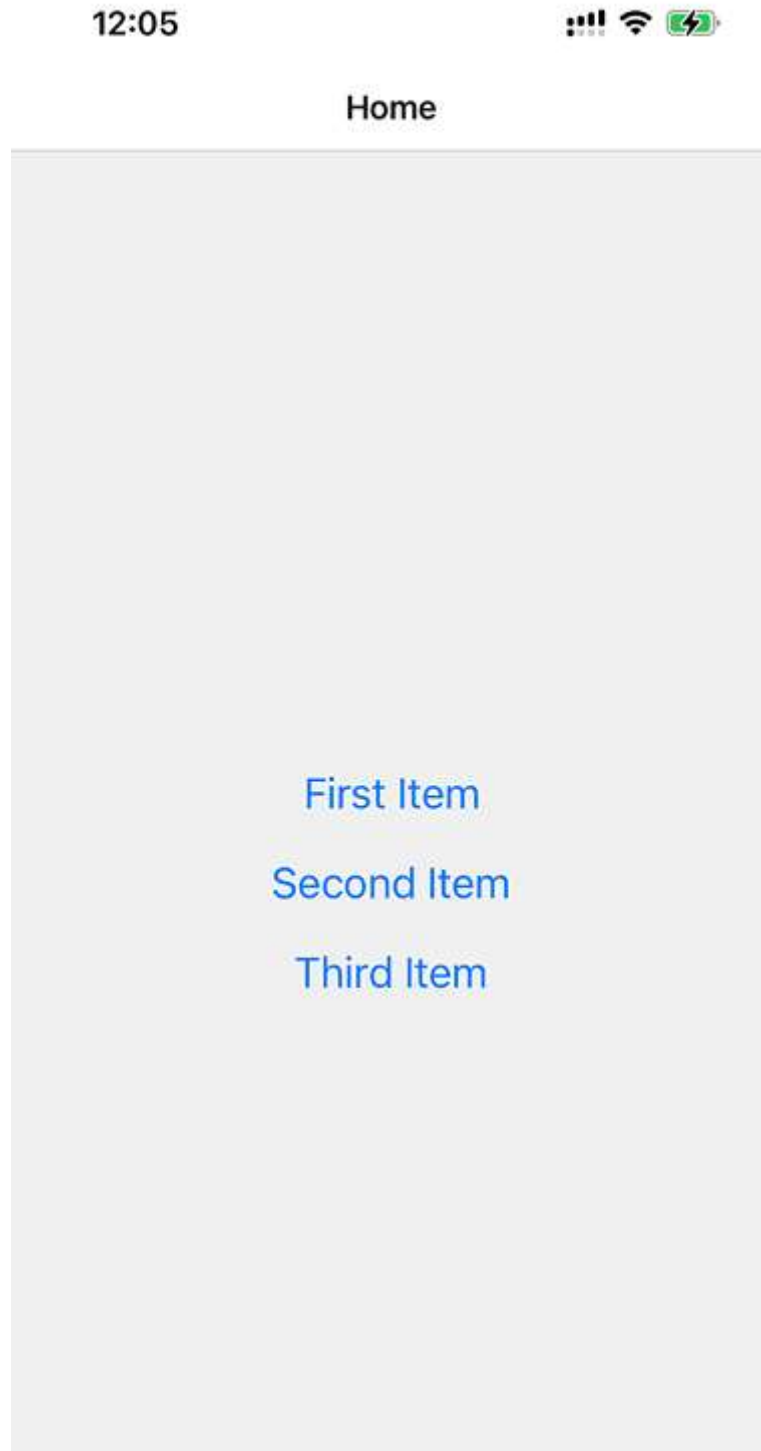




Figure 19.5: The Home screen

Now there is header text in the navigation bar, which is set by the name property in the `Screen` component.

Next, try pressing the **First Item** button:

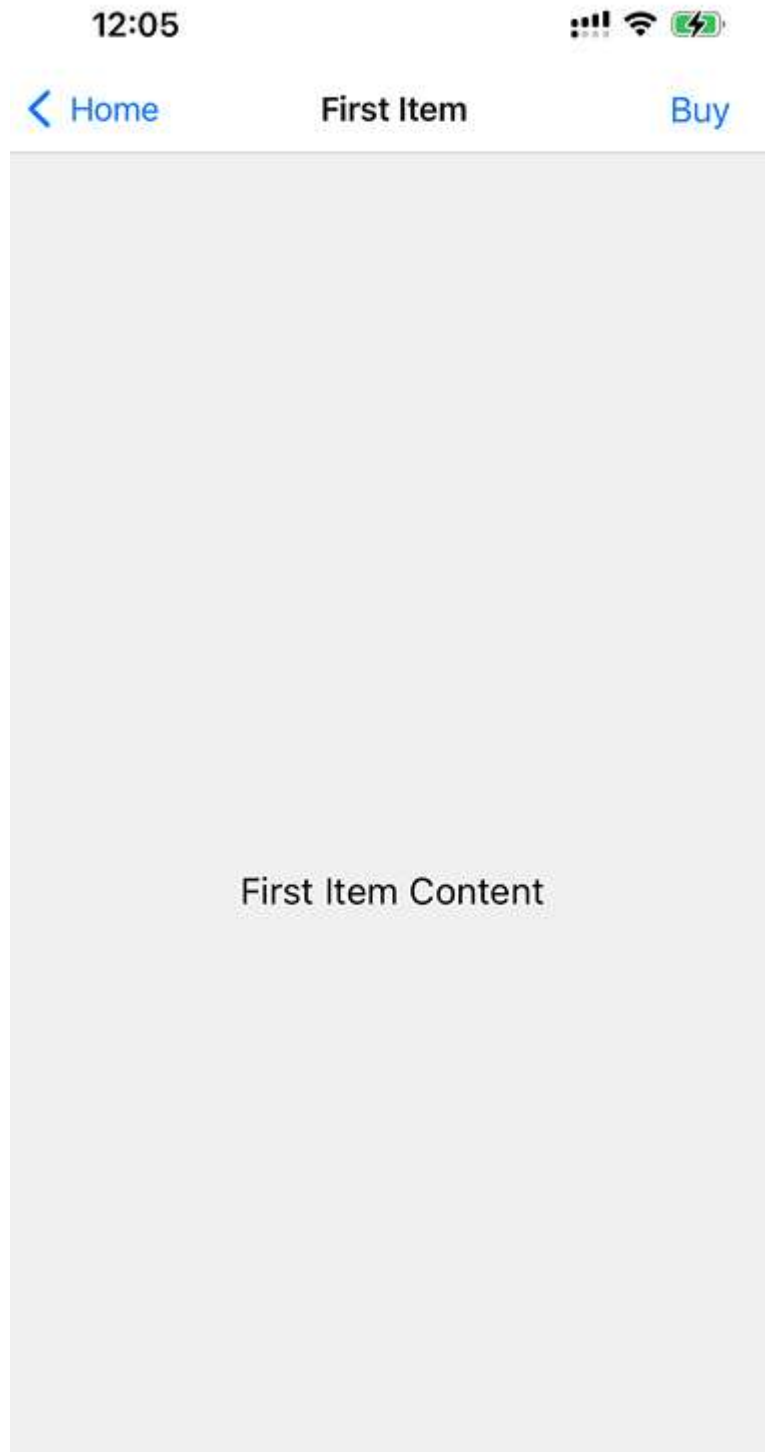




Figure 19.6: The First Item screen

The title in the navigation bar is set based on the `title` parameter that's passed to the `Details` component using the `navigation.setOptions()` method. The **Buy** button that's rendered on the right side of the navigation bar is rendered by the `options` property in the `Screen` component placed in the `App` component. It's enabled because the `stock` parameter value is `1`.

Now, try returning to the **Home** screen and pressing the **Second Item** button:

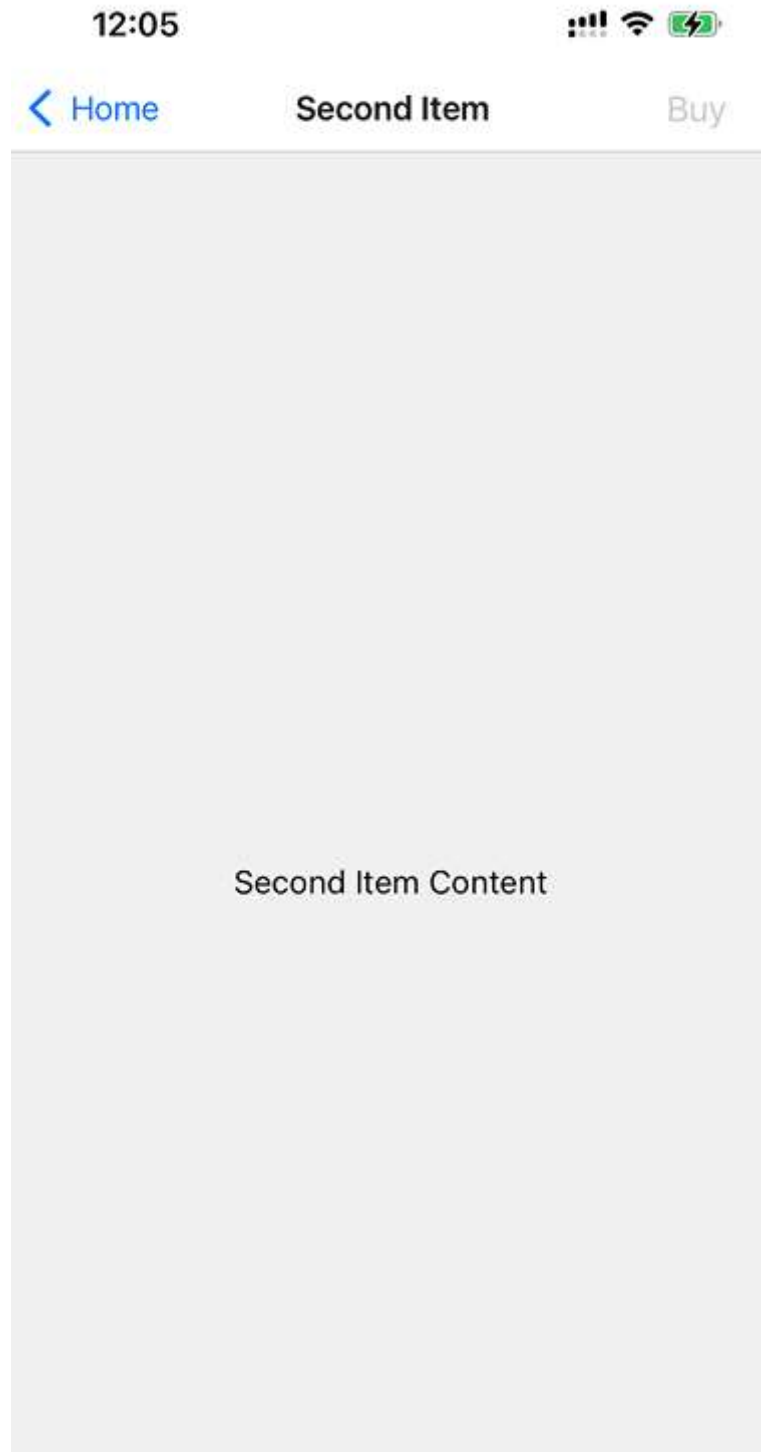




Figure 19.7: The Second Item screen

The title and the page content both reflect the new parameter values passed to `Details`, but so does the `Buy` button. It is in a disabled state because the stock parameter value was `0`, meaning that it can't be bought.

Now that you've learned how to use navigation headers, in the next section, you'll learn about tab and drawer navigation.

Tab and drawer navigation

So far in this chapter, each example has used `Button` components to link to other screens in the app. You can use functions from `react-navigation` that will create **tab** or **drawer** navigation for you automatically based on the screen components that you give it.

Let's create an example that uses bottom tab navigation on iOS and drawer navigation on Android.

You aren't limited to using tab navigation on iOS or drawer navigation on Android. I'm just picking



these two to demonstrate how to use different modes of navigation based on the platform. You can use the exact same navigation mode on both platforms if you prefer.

For this example, we need to install a few other packages for tab and drawer navigators:

```
npm install @react-navigation/bottom-tabs @react-navigation/drawer
```

Also, the drawer navigator requires some native modules. Let's install them:

```
npx expo install react-native-gesture-handler react-native-reanimated
```

Then, add a plugin to the `babel.config.js` file. As a result, the file should look like the following:

```
module.exports = function (api) {  
  api.cache(true);  
  return {  
    presets: ["babel-preset-expo"],  
    plugins: ["react-native-reanimated/plugin"],  
  };  
};
```

Now, we are ready to continue coding. Here's what the `App` component looks like:

```
const Tab = createBottomTabNavigator<Routes>();
const Drawer = createDrawerNavigator<Routes>();
export default function App() {
  return (
    <NavigationContainer>
      {Platform.OS === "ios" && (
        <Tab.Navigator>
          <Tab.Screen name="Home" component={Home} />
          <Tab.Screen name="News" component={News} />
          <Tab.Screen name="Settings" component={Settings} />
        </Tab.Navigator>
      )}
      {Platform.OS === "android" && (
        <Drawer.Navigator>
          <Drawer.Screen name="Home" component={Home} />
          <Drawer.Screen name="News" component={News} />
          <Drawer.Screen name="Settings" component={Settings} />
        </Drawer.Navigator>
      )}
    </NavigationContainer>
  );
}
```

Instead of using the `createNativeStackNavigator()` function to create your navigator, you're importing the `createBottomTabNavigator()` and `createDrawerNavigator()` functions:

```
import { createDrawerNavigator } from "@react-navigation/drawer";  
import { createBottomTabNavigator } from "@react-navigation/bottom-tabs";
```

Then, you're using the `Platform` utility from `react-native` to decide which navigator to use. The result, depending on the platform, is assigned to `App`. Each navigator contains the `Navigator` and `Screen` components, and you can pass them to your `App`. The resulting `tab` or `drawer` navigation will be created and rendered for you.

Next, let's take a look at the `Home` screen component:

```
export default function Home() {  
  return (  
    <View style={styles.container}>  
      <Text>Home Content</Text>  
    </View>  
  );  
}
```

The `News` and `Settings` components are essentially the same as `Home`. Here's what the bottom `tab` navigation looks like on iOS:

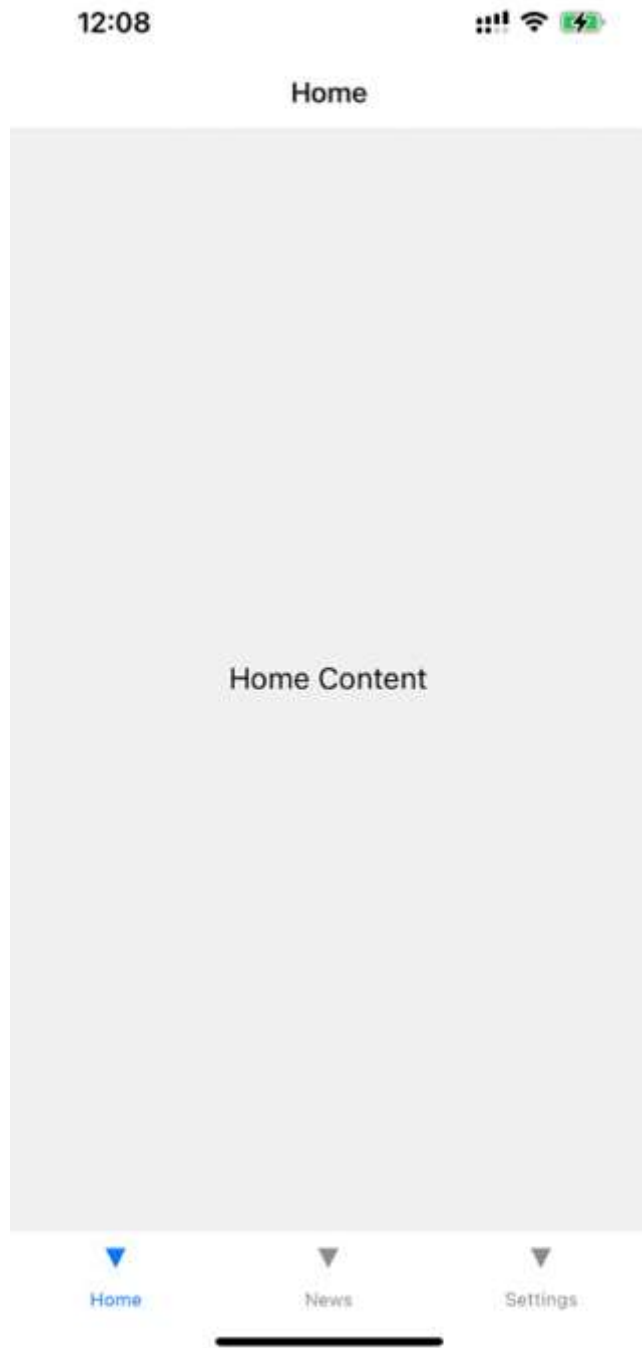


Figure 19.8: The tab navigator

The three screens that make up your app are listed at the bottom. The current screen is marked as active, and you can click on the other tabs to move around.

Now, let's see what the `drawer` layout looks like on Android:

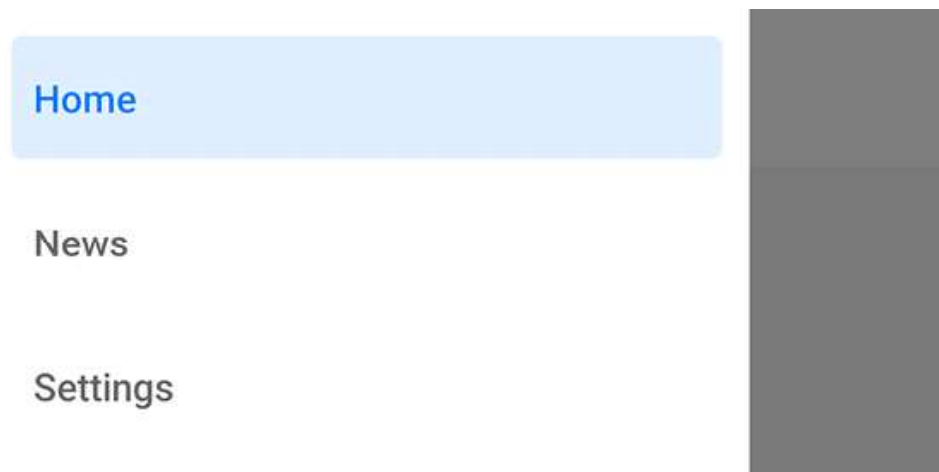


Figure 19.9: The drawer navigator

To open the `drawer`, you need to swipe from the left side of the screen. Once it's open, you'll see buttons that will take you to the various screens of your app.



Swiping the `drawer` open from the left side of the screen is the default mode. You can configure the



`drawer` to swipe open from any direction.

Now, you've learned how to use `tab` and `drawer` navigation. Next, we'll explore the approach of how to define navigation based just on files.

File-based navigation

In this section, we will talk about **Expo Router**, a file-based router works in a similar way to routing in Next.js. To add a new screen, you just need to add a new file to the `app` folder. It's built on top of React Navigation, so the routes have the same options and parameters.



For more information and details about Expo Router, take a look at this link:

<https://docs.expo.dev/routing/introduction/>

To try it out, we will install a fresh project using:

```
npx create-expo-app -template
```

To install the project with Expo Router ready, we just need to choose the `Navigation (TypeScript)` template:

```
Blank
Blank (TypeScript)
> Navigation (TypeScript) - File-based routing with TypeScript enabled
Blank (Bare)
```

When the installation is finished, you will find the `app` folder for the project. This folder will be used for all your screens. Let's try to replicate the example from the *The basics of navigation* section. First of all, we need to create `_layout.tsx` inside the `app` folder. That file is working as a `root` layer of our `app`. This is how it looks:

```
import { Stack } from "expo-router";
export default function RootLayout() {
  return <Stack />;
}
```

Then let's create the `index.tsx` files that will contain the `Home` screen. It has few differences compared to `_layout.tsx`, so let's take a look:

```
import { Link } from "expo-router";
export default function Home() {
  return (
    <View style={styles.container}>
      <StatusBar barStyle="dark-content" />
    </View>
  );
}
```

```
    <Text>Home Screen</Text>
    <Link href="/settings" asChild>
      <Button title="Settings" />
    </Link>
  </View>
);
}
```

As you can see here, we don't use a `navigation` prop. We are instead using a `Link` component that accepts `href` props, just like a web page. Clicking on that button takes us to the `Settings` screen.

Let's create the `settings.tsx` file:

```
import { Link } from "expo-router";
export default function Settings() {
  return (
    <View style={styles.container}>
      <StatusBar barStyle="dark-content" />
      <Text>Settings Screen</Text>
      <Link href="/" asChild>
        <Button title="Home" />
      </Link>
    </View>
  );
}
```

Here, we use the same approach as the `index.tsx` files, but in `Link`, we set `href` to `“ / ”`.

This is how easily we can define screens in a declarative way, and the URL approach to navigating between screens works out of the box. Also, one benefit we get here is that **deep linking** also works out of the box; with this method, we can open specific screens using app links.

Now you know how to use file-based routing, which can improve your experience of developing mobile apps, especially with a web-based mindset of URLs and linking.

Summary

In this chapter, you learned that mobile applications require navigation, just like web applications do. Although they are different, web application and mobile application navigation have enough conceptual similarities that mobile app routing and navigation don't have to be a nuisance.

Older versions of React Native attempted to provide components to help manage navigation within mobile apps, but they never really took hold. Instead, the React Native community has dominated this area. One example of this is the `react-navigation` library: the focus of this chapter.

You learned how basic navigation works with `react-navigation`. You then learned how to control `header` components within the navigation bar. Next, you learned about the `tab` and `drawer` navigation components. These two navigation components can automatically render the navigation buttons for your app based on the screen components. You also learned how to work with the file-based Expo Router.

In the next chapter, you'll learn how to render lists of data.